

Burst Splitter Internal Logic

Overview

The Burst Splitter is a two-stage pipelined block shared across all N_CLIENTS AXI clients. It sits between the AW/AR FIFO and the Packet Formation Unit. Its function is to convert validated AXI burst requests into normalized per-beat packets, splitting bursts that cross DRAM row boundaries into two sub-transactions.

Input: AW/AR FIFO output — ADDR, AXID, AXLEN, AXSIZE, AXBURST

Output: Per-beat context (addr, tag, pkt_type, beat_cnt) → Packet Formation Unit via pkt_valid

Stage 1 — Decode & Compute (1 cycle, registered output)

Stage 1 runs combinatorially and captures all results into the S1→S2 pipeline register on the next clock edge. Three sub-blocks operate in parallel:

Burst Type Decoder

- Input: AXBURST[1:0] from FIFO
- Output: is_wrap / is_incr flag → pipeline register
- FIXED burst type is not supported (design-time assertion)

Row Boundary Checker

- Inputs: ADDR, AXLEN, AXSIZE from FIFO
- Computes: addr_start + burst_bytes vs ROW_MASK
- Outputs: split_needed flag, row_boundary address → Split Point Calc and Beat Count Calc

WRAP Address Generator

- Inputs: ADDR, AXLEN, AXSIZE from FIFO
- Active only when is_wrap = 1
- Outputs: wrap_boundary, beat_addrs[0..N] → pipeline register
- Maximum 16 entries (AXI4 WRAP burst max AXLEN = 15)

Split Point Calc

- Inputs: row_boundary from Row Boundary Checker; ADDR from FIFO
- Computes: addr_b = row_boundary, beat_cnt_b = total_beats - beat_cnt_a
- Output: addr_b, beat_cnt_b → pipeline register

Beat Count Calc

- Inputs: `row_boundary` from Row Boundary Checker; `AXLEN` from FIFO
- Computes: `beat_cnt_a = beats_to_row_end`, `total_beats = AXLEN + 1`
- Output: `beat_cnt_a`, `total_beats` → pipeline register

S1→S2 Pipeline Register (clocked)

```
{ addr_a, beat_cnt_a, addr_b, beat_cnt_b, split_needed, is_wrap, beat_addrs[], axid,
  pkt_type, tag }
```

Stage 2 — Emit & Track

Stage 2 is multi-cycle when a split is needed. It stalls Stage 1 (via `s1_busy`) while emitting sub-transaction B.

Emit FSM

- IDLE → EMIT_A
- EMIT_A → IDLE (split_needed = 0)
- EMIT_A → EMIT_B (split_needed = 1)
- EMIT_B → IDLE
- Hold current state if `pkt_ready = 0`
- Asserts `s1_busy` during EMIT_B to stall Stage 1

Sub-transaction A Mux

- Selects: `addr_a`, `beat_cnt_a`, `tag`, `pkt_type` from pipeline register
- Forwards to Pkt Assembler on cycle N

Sub-transaction B Mux

- Selects: `addr_b`, `beat_cnt_b`, same `tag`, `pkt_type` from pipeline register
- Forwards to Pkt Assembler on cycle N+1

Beat Tracker

- Maintains `beat_cnt_reg`; decrements on `pkt_accept`
- Asserts `last_beat` flag when count reaches zero
- Output: `last_beat` → Pkt Assembler

WRAP Beat Address Table

- Up to 16-entry register file, written once on S1→S2 pipeline register transfer
- Input: `beat_addrs[]` from S1→S2 pipeline register
- Read sequentially by `beat_idx` counter
- Output: current beat address → Pkt Assembler

Pkt Assembler

- Inputs: `addr`, `tag`, `pkt_type`, `beat_cnt` from Sub-txn A/B mux; `last_beat` from Beat Tracker; beat address from WRAP beat addr table
- Assembles normalized read (42-bit) or write (618-bit) packet
- Asserts `pkt_valid` → Packet Formation Unit

Backpressure & Control Signals

Signal	Direction	Description
<code>s1_busy</code>	Emit FSM → AW/AR FIFO	Stalls Stage 1 from consuming next FIFO entry during <code>EMIT_B</code>
<code>pkt_ready</code>	Packet Formation Unit → Emit FSM	Back-pressure from downstream; FSM holds state when low
<code>pkt_valid</code>	Pkt Assembler → Packet Formation Unit	Indicates normalized packet ready for consumption

Three Paths Through the Splitter

Path A — No split (INCR, no row crossing)

`split_needed` = 0. Stage 2 emits one packet (`EMIT_A` → `IDLE` in 1 cycle). `s1_busy` never asserted. Throughput: 1 transaction / cycle.

Path B — Row-boundary split (INCR, crosses boundary)

`split_needed` = 1. Stage 1 computes `addr_b`, `beat_cnt_b`. Stage 2: `EMIT_A` cycle N, asserts `s1_busy`, `EMIT_B` cycle N+1. Throughput: 1 transaction / 2 cycles.

Path C — WRAP burst

WRAP address generator fills `beat_addrs[]`. Stage 2 iterates via WRAP beat addr table indexed by `beat_idx`. May also split at row boundary.

Internal State

Field	Description
<code>base_addr</code>	Base address of current burst
<code>beat_idx</code>	Current beat index within burst
<code>beats_left</code>	Remaining beats to emit
<code>burst_type</code>	INCR or WRAP
<code>burst_len</code>	Total beats (<code>AXLEN</code> + 1)
<code>txn_tag</code>	Allocated transaction tag [<code>AXID</code> [5:0] + <code>seqnum</code> [1:0]]

Timing Notes

- Stage 1 combinatorial depth must close timing in 1 cycle. WRAP address generator is the critical path — limited to $AXLEN \leq 15$ for WRAP bursts (AXI4 constraint).
- `S1→S2` register clocked at `posedge clk`.
- `pkt_ready` = 0 holds the Emit FSM in its current state; `s1_busy` remains asserted until `IDLE` is reached.

- Tag Allocator must have a valid tag before Stage 1 can proceed — Tag Allocator stall folds into `s1_busy`.
- Error path (`pkt_type = Err_write / Err_read`): set in Stage 1, carried through pipeline register unchanged. Both sub-transactions on a split carry the same `pkt_type`.